Sthira: What We Are Building, Why It Is Different, and Where It Goes

Team ECR: Ojas Mutreja, Vidit Dang, Danica Gupta

Hacktoberfest Hack Day Bengaluru 2026 · Track 2 · 02

2026-10-18

Part 1 The short answer

Sthira (स्थिर, Sanskrit for *steady, firm*) remembers what you decided while working with AI, and speaks up when something is about to undo it.

It reads your AI history: ChatGPT and Claude exports, Claude Code sessions, meeting transcripts. A small open-weight model running on your own laptop (Qwen3) pulls out the decisions you made, with the reason and a word-for-word quote from where you made them. Those go into a ledger you own. When an AI agent later proposes something that goes against one of them, Sthira warns you and shows you exactly where you decided it. When Sthira gets something wrong, you correct it, and it learns.

HEADLINE Your AI forgets what you decided. Sthira doesn't.

- **Live demo:** https://sthira-mocha.vercel.app
- **Code (MIT):** https://github.com/HelloOjasMutreja/Sthira
- **Track:** Best Open-Source AI Project, problem statement O2 (Adaptive AI with Memory and Self-Correction)

Part 2 The problem we kept running into

Millions of people now do real work with AI every day. Decisions get made inside those conversations: which database, which login flow, what ships first, what we tried and why it failed. Then the conversation ends, and the decision lives nowhere useful.

Three things make this worse every month:

1. **Every tool starts from zero.** ChatGPT, Claude, Cursor and Claude Code each keep their own memory, if any. Moving between them is a one-time copy at best.

2. **Agents act on stale or missing context.** A coding agent that never saw your March decision will happily “fix” performance by migrating to the database you rejected. A 2026 study of coding agents found that a stale instruction file can be worse than no file at all.
3. **Nobody re-reads the history.** Chats, summaries and session logs pile up. People re-litigate decisions in week three because no one remembers what was agreed.

What people say they want lines up with this. In the Stack Overflow 2026 survey, only 7% of developers trust AI for important decisions without being able to check it, but 48% do when the output is easy to verify. Making past decisions visible and citable is exactly that kind of checkability.

Part 3 What we built today

Sthira is a working tool, not a mockup. Everything below runs.

Table 1. What Sthira does

Capability	How it works
Reads your AI history	Parsers for ChatGPT and Claude.ai exports, Claude Code session logs and meeting transcripts
Extracts what you settled	A local open-weight model reads each conversation and proposes decisions, open questions, preferences, commitments and lessons, each with its reason and a verbatim quote
Refuses to make things up	Every quote is checked against the source text. If it is not really there, the entry is dropped
Never forgets history	Nothing is deleted. When you change your mind, the old decision is marked superseded and points to the new one
Speaks up	`sthira check`, `sthira scan` (a whole agent session), a live Claude Code hook, an MCP tool, and the web app all return a cited warning
Learns from you	Rejected extractions and corrected warnings become rules that every future run follows

Capability	How it works
Every agent sees it	`sthira sync` writes the settled decisions into `AGENTS.md / CLAUDE.md`, and the MCP server serves them to any AI tool

The demo story

Acme Notes is a fictional note-taking app. In March, in a ChatGPT chat, the team decided on Postgres, not MongoDB, because they need joins. Today, a Claude Code session tries to fix slow search by migrating to MongoDB and adds a password login on the side.

Sthira catches both. It quotes the March chat (“Ok, decided: we go with Postgres, not MongoDB”), names the date and the conversation, and asks whether to keep the decision or replace it. If you replace it, the old one stays in the ledger as history.

Part 4 How this is different, after all the research

We did two rounds of prior-art research (about 80 open-source projects, the big platforms, enterprise tools and recent papers) before writing a line of code. The honest picture: almost every piece of this idea already exists somewhere. What does not exist is the combination.

Table 2. Where the existing tools stop

Category	Examples	What they do	What they don’t do
Platform memory	ChatGPT, Claude, Gemini, Copilot	Remember you inside one product	Share with other tools; treat decisions as objects; detect contradictions
Memory layers	Mem0, Supermemory, Letta, Cognee, claude-mem	Store facts about you for agents	Track decisions with reasons and history; warn when work goes against them
History search	claude-code-history-viewer, cass, chat exporters	Let you search old sessions	Pull out what was decided; check new work

Category	Examples	What they do	What they don't do
Decision tools	projectmem, Axiom Hub, Sentra, Town	Log decisions per repo or per team, mostly cloud	Read your personal AI chats across vendors; run on a local open model
Integration hubs	Zapier MCP, n8n, MCP gateways	Connect thousands of apps	Remember anything at all

FINDING Nobody combines four things: sources from every AI vendor you use, decisions with reasons and a citable source, contradiction warnings from a local open-weight model, and an open format any tool can read.

So the claim we make is narrow on purpose: Sthira is the first open, local, cross-vendor ledger of what you have settled. We do not claim to be the first memory tool, or the first to detect a contradiction. Both would be false, and judges who know the space would know.

What we learned from other projects' mistakes

Reading the issue trackers of popular memory tools shaped the design more than any paper did:

- One public audit of a popular memory layer found 97.8% of stored memories were junk. **Our answer:** the extractor is told that returning nothing is normal, and quotes must be verified.
- Small local models break JSON output. **Our answer:** schema-constrained decoding, with the reasoning fields placed before the verdict field.
- Hyped benchmark claims caused public backlash. **Our answer:** a small, reproducible eval with its limits written down.
- Memory poisoning is real (it is on the OWASP list for agents). **Our answer:** new entries stay *quarantined* until a human settles them.
- Several "local" tools quietly drifted to cloud and telemetry. **Our answer:** MIT licence, no telemetry, no network calls except to the model you configure.

Part 5 How it meets O2, with numbers

O2 asks for AI that retains context, learns from feedback, and identifies and corrects its own mistakes. We built each of those and measured them on our demo scenario with `sthira demo` and `scripts/eval_checks.py`.

Table 3. Extraction quality, before and after learning from corrections

Model (on a laptop CPU)	Precision before corrections	Precision after corrections	Recall
qwen3:4b	0.60	0.71	0.80
qwen3:8b	1.00	1.00	0.80

Table 4. Contradiction checks (3 real conflicts, 3 traps)

Pipeline on qwen3:4b	Correct
Single pass	4 of 6 (flagged “add a cache” as a contradiction)
Single pass plus a self-correction critic	6 of 6

The critic is the part we are proudest of. When the first pass claims a contradiction, a second pass asks one concrete question: does the new work stop using, migrate away from, or go against what was chosen? Our first version of the critic answered before it reasoned, and it talked itself out of real contradictions. Putting the reasoning fields first fixed it. On a 4B model, the order of fields in a JSON schema mattered.

> FINDING Learning from corrections matters most on small models. That is the right place for it to matter, because small models are what make running privately on a laptop possible.

These are smoke-test numbers from one scenario we built ourselves. They show the mechanism works. They are not a benchmark, and the eval README says so.

Part 6 The super tool

Today's Sthira is the core of a much bigger idea. Integration hubs connect tools but remember nothing. Platform memories remember, but only for themselves. Sthira sits between them and holds **meaning**: what is decided, why, what is still open, and what changed.

Three kinds of connection

Inputs (everything that feeds it). AI chats and coding agents first. Then meetings through local Whisper transcription, voice notes from your phone, notes apps such as Obsidian and Notion, email and calendar (for commitments), git and pull requests (what actually happened), browser research (the sources behind a decision), and group chats in Slack or Discord.

The core (what holds it together). An open, plain-text ledger format, readable by people and any tool, in the way calendars agreed on one file format. A local open model as the judge. Every write confirmed by you.

Outputs (everything that obeys it). Every AI assistant over MCP. Instruction files that are always fresh. Git hooks and CI that flag a change breaking a settled decision. An editor that shows the decision behind any line of code. A calendar brief before each meeting. Tasks that close themselves when a commitment is done.

What happens when it is all connected

- **A why behind everything.** Any line of code or paragraph of a document can answer "why is it like this?" by tracing back to the decision.
- **Agents that ask instead of guessing.** On anything still open, an agent asks you rather than inventing an answer.
- **Decisions that reopen themselves.** A decision records its assumptions. When one breaks ("the library you chose was archived yesterday"), Sthira suggests reopening it.
- **Your settled self travels with you.** Switch from ChatGPT to Claude to a local model, and your decisions come along. Memory stops being lock-in.

- **Projects publish their ledgers.** An open-source project publishes “we will not add feature X, because Y”. Every contributor’s agent reads it before opening a PR, which goes straight at the low-effort-PR problem that changed Hacktoberfest itself.
- **Disagreements surface early.** Two teammates’ ledgers disagree, and it shows up before anyone builds on the wrong one.

Table 5. How it grows

Stage	What ships
Today (hackathon)	Core ledger, 4 input types, CLI, MCP server, Claude Code hook, web app, self-correcting checks, eval
v1	Voice notes, Obsidian, git hooks, broken-assumption alerts, a loose-ends radar for promises and open questions
v2	Sharing between people, published project ledgers
Long term	The open ledger format becomes a standard other tools adopt

Every new source or output is a small, separate file, which makes it an easy first contribution. We have already opened good-first issues for Gemini, Cursor, subtitle files and Cursor rules.

Part 7 The pitch

One line

Your AI forgets what you decided. Sthira doesn’t.

Thirty seconds

We all make decisions with AI now, across ChatGPT, Claude and coding agents. Weeks later an agent cheerfully undoes one, because nothing remembers what was settled or why. Sthira reads your AI history with an open-weight model on your own laptop, keeps a ledger of your decisions with their reasons and sources, and warns

any agent that tries to go against them. It learns from your corrections, and it never deletes your history.

Two minutes, for the judges

1. **The problem (15 seconds).** Decisions made with AI vanish. Platform memories are siloed and none of them track decisions.
2. **What it is (15 seconds).** Show the ledger: a decision, its quote, its source chat and date.
3. **The moment (30 seconds).** Run the MongoDB example. Sthira flags it and quotes the March chat. It also caught the agent adding password login.
4. **It learns (35 seconds).** Show the rejected “kitten named Miso” entry, the precision going from 0.60 to 0.71, and the critic taking checks from 4 of 6 to 6 of 6.
5. **Every agent sees it (15 seconds).** Show the `AGENTS.md` block and the MCP tools.
6. **Close (10 seconds).** Open source, local-first, open-weight. Nothing deleted, everything cited.

Questions we expect

- **Why not just use ChatGPT or Claude memory?** They are siloed per vendor, other tools cannot read them, and none of them treat decisions, reasons or contradictions as first-class.
- **How do you stop made-up memories?** Every quote must exist word for word in the source, and nothing becomes settled until a person confirms it.
- **How is this different from Mem0?** Memory layers store facts about you. Sthira stores decisions with their reasons and history, and checks new work against them.
- **Why open-weight?** Your AI history is private. A 4B model on a laptop is enough to read it, so nothing has to leave your machine.

Part 8 Honest limits

- The eval is one scenario we built ourselves. It proves the mechanism, not general accuracy.
- On a laptop CPU, a check takes about 20 seconds with qwen3:4b. A GPU or a hosted open-weight model makes it near instant.
- Automatic extraction will always miss some decisions and invent some. That is why the human stays in the loop and why every entry carries its quote.
- The hosted demo uses an open-weight model through a gateway. If that gateway is unavailable, the built-in examples fall back to verdicts recorded from a real local run, and the page says so.
- Big platforms could add portable memory with decision tracking. Our defence is being open, local and cross-vendor, which their business models discourage.

Part 9 Where to look

- **Live demo:** https://sthira-mocha.vercel.app
- **Repository:** https://github.com/HelloOjasMutreja/Sthira
- **Run the proof yourself:** `uv run sthira demo` and `uv run python scripts/eval_checks.py`
- **Research behind the decisions:** `docs/research/` in the repository
- **Team ECR:** Ojas Mutreja (lead), Vidit Dang, Danica Gupta